

COMPARISON OF COMBINATORIAL AND OPTIMIZATION METHODS FOR GRAPH PROBLEMS

BY
JASMINE SALDANO

SENIOR THESIS

Submitted in partial satisfaction of the requirements for the degree of

BACHELOR OF SCIENCE

in

MATHEMATICS

in the

COLLEGE OF LETTERS AND SCIENCE

of the

UNIVERSITY OF CALIFORNIA, DAVIS



Approved:

Contents

Acknowledgements	2
Introduction	3
1 Background	5
1.1 Graph Theory	5
1.2 Combinatorial Algorithms	8
1.2.1 Dijkstra’s Algorithm	8
1.2.2 Bellman-Ford Algorithm	10
1.2.3 Comparison of Dijkstra and Bellman-Ford	12
1.3 Linear Programming and Optimization	13
2 Experimental Design	17
2.1 PassageMath Environment	17
2.2 Algorithm Implementations	17
2.3 Runtime and Scalability Comparisons	17
3 Results and Discussion	18
4 Future Work	19
Bibliography	19

Acknowledgements

Introduction

Graph problems arise in many areas of mathematics and computer science, particularly in combinatorics, optimization, artificial intelligence, machine learning, transportation systems, and communication networks. Among the most fundamental graph problems are shortest path, routing, and network flow, which model tasks such as GPS navigation, resource allocation, and path planning for intelligent systems. Many of these problems can be solved through either combinatorial graph algorithms or optimization-based methods.

In this thesis, we compare combinatorial and optimization methods for solving graph problems, with a particular focus on shortest path formulations. We examine classical combinatorial algorithms including Dijkstra and Bellman-Ford, and compare them with linear programming and flow-based optimization formulations. Our goal is to study both the mathematical structure and computational behavior of these approaches through implementations and experiments in SageMath and PassageMath.

Computational comparisons between algorithms are important because algorithm performance can depend heavily on graph properties such as graph size, density, structure, and weighting conditions. In particular, optimization-based methods may provide flexibility for more complex constraints, while combinatorial algorithms often offer strong efficiency on classical graph problems. Through computational experiments, we analyze runtime behavior, scalability, and sensitivity to graph structure for several graph families, including randomly generated graphs and structured examples from graph theory. With this, we aim to better understand the strengths and limitations of combinatorial and optimization-based techniques on various classes of graphs.

The SageMath and PassageMath software systems provide a useful environment for this study due to their extensive graph theory, combinatorics, and optimization libraries. In particular, SageMath contains existing implementations of shortest path algorithms, network flow routines, and linear programming solvers, making it possible to compare algorithmic and optimization-based techniques within a unified computational framework.

In Part 1, we review background material from graph theory, combinatorics, and linear programming. We introduce graph problems such as shortest path and network flow, discuss combinatorial algorithms, and present linear programming formulations relevant to

graph problems. In Part 2, we discuss SageMath and PassageMath software environment, including the implementations used in this project. In Part 3, we present computational experiments comparing combinatorial and optimization approaches on several graph families. This includes analyzing their runtime behavior, scalability, and sensitivity to graph structure. Finally, we conclude with observations, limitations, and possible directions for future work involving more advanced graph optimization problems.

Chapter 1

Background

Graph theory, combinatorial optimization, and linear programming are closely connected areas of mathematics and computer science that appear in many real-world computational problems. Graphs provide mathematical models for representing relationships and networks, where vertices represent objects and edges represent connections between them. The first paper on graph theory was written in 1736 by the famous Swiss mathematician Leonhard Euler, which dealt with the well-known Königsberg bridge problem [1]. Graph theory has historical roots in puzzles and games, but today it provides a natural framework for investigations in many disciplines, including networks, chemistry, psychology, social science, and genetics. Graphs are some of the most useful models in computer science, becoming especially valuable in routing, artificial intelligence, and machine learning.

1.1 Graph Theory

A graph G , also called a simple graph, is composed of two objects. It has a finite set of elements $V = \{a, b, c\}$ called vertices or nodes, and a set E of pairs of distinct vertices called edges. Every node v is usually depicted as a point in the graph, and every edge e is usually depicted as a line that connects exactly two nodes [4]. A graph whose vertex set is V and whose edge set is E is denoted by

$$G = (V, E).$$

An edge is a tuple (u, v) that represents the link between nodes u and v . The number of edges connected to a node v is called the degree of v [4]. Generally, a simple graph is assumed to have at most one edge connecting any pair of vertices, and no vertex is connected to itself [1]. In an undirected graph, all edges can be traversed in both directions, whereas an edge (u, v) of a directed graph can only be traversed from u to v [4].

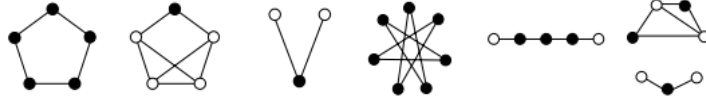


Figure 1.1: Examples of simple graphs.

In Figure 1.1 are various examples of simple graphs, where some graphs contain an odd number of nodes and black circles indicate nodes of even degree [1].

A walk in a graph G is a sequence of one or more nodes $v_0, v_1, \dots, v_k$ such that any two consecutive nodes in the sequence are adjacent. The shortest possible walk consists of a single node v_0 . If each edge is assigned a nonnegative real number, then this value represents the length or weight of the edge.

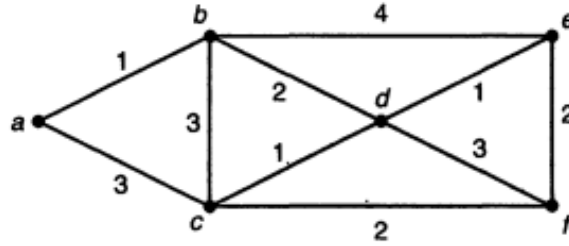


Figure 1.2: Weighted graph example.

In the weighted graph depicted in Figure 1.2, the numbers beside edges denote edge lengths, also called the weights [1]. One example of a walk joining vertices a and e is a, b, d, e , which has length 4 after adding the edge lengths.

A path $P = \{s, \dots, u, \dots, v, \dots, t\}$ in a graph is a sequence of nodes connected by edges from a source node s to a target node t [4]. Additionally, a path is a walk that does not repeat any vertices. The first and last vertices are called endpoints of the path, and connecting them forms a cycle such as in Figure 1.3. A path $P' = \{u, \dots, v\}$ is a subpath of P if its sequence of nodes and edges are all contained within the same order in P [4]. The weight of a path $w(P)$ is the sum of all the weights associated to the edges involved in the path [4].

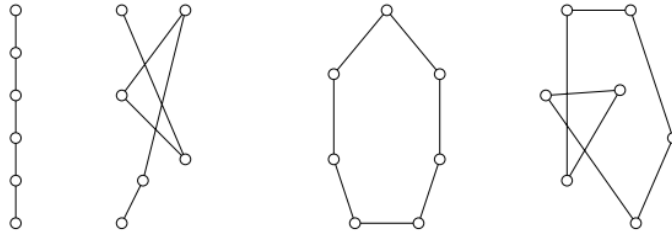


Figure 1.3: Examples of paths and cycles.

A graph G is connected if there exists a path between every pair of vertices. Graphs are either connected or disconnected, but some connected graphs are "more connected" than others [1]. This can be measured by how many vertices or edges must be removed in order to disconnect the graph. A graph that is not connected can be decomposed into connected components, also called maximal connected subgraphs [1]. Connected components capture the idea that a graph may consist of multiple independent regions of connectivity.

Theorem 1.1.1. [1] *Let $G = (V, E)$ be a graph. Then the vertex set V can be uniquely partitioned into nonempty subsets $V_1, V_2, \dots, V_k$ such that:*

1. *The induced subgraph $G_i = (V_i, E_i)$ is connected for each i .*
2. *If $i \neq j$, then there is no path in G between any vertex in V_i and any vertex in V_j .*

Each connected component acts as an independent subgraph in which standard graph problems are well-defined. In particular, vertices belonging to different components are not reachable from one another. This has direct implications for shortest path problems. If no path exists between vertices s and t , then no finite-length path can be formed between them, and the distance between such vertices is typically considered infinite or undefined. As a result, shortest path algorithms are naturally applied within connected components.

For vertices s and t within the same connected component, the shortest path between them is a path of minimum total weight among all paths connecting s to t . The corresponding shortest path distance is defined as

$$d(s, t) = \min_{P: s \rightarrow t} \sum_{e \in P} w(e),$$

where the minimum is taken over all paths P from s to t , and $w(e)$ denotes the weight of edge e . This is also called the minimum distance.

Beyond shortest path problems, graphs can also model the movement of quantities through a network. Flow problems model the movement of quantities through a directed graph, where each edge is assigned a nonnegative capacity representing the maximum allowable transfer. Unlike shortest path problems, which seek a single minimum-weight path between two vertices, flow problems distribute quantity across multiple paths between a source s and a sink t . These models extend weighted graph structures and motivate optimization-based formulations of graph problems discussed in Section 1.3.

In graphs that model physical situations, such as the paths in the maps of Figure 1.4, some edges are considered more costly than others. As such, edges are often assigned weights that represent quantities such as distance, time, cost, or energy. These weights allow graphs

to encode optimization problems over networks, where the goal is to compare and select among competing routes or patterns of connectivity.

As a result, weighted graphs are fundamental in providing the underlying structure for both shortest path and flow problems, since both rely on evaluating and optimizing paths in a network. This connection between graph structure and optimization motivates the algorithmic and linear programming approaches studied in later sections.

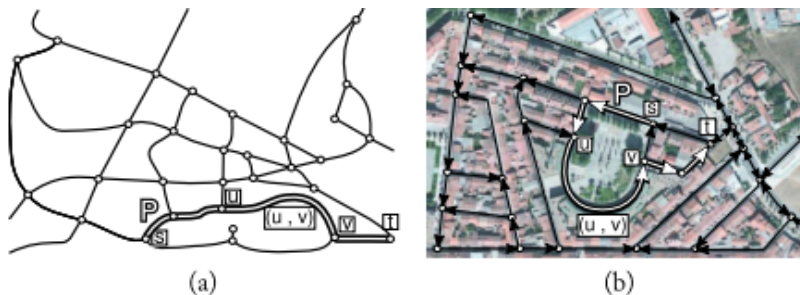


Figure 1.4: Examples of real-world paths in (a) an undirected graph and (b) a directed graph

1.2 Combinatorial Algorithms

Many graph problems can be solved using combinatorial algorithms, which operate directly on the discrete structure of a graph. These algorithms systematically explore vertices and edges according to specific rules and are widely used in computer science, optimization, and artificial intelligence.

1.2.1 Dijkstra's Algorithm

As explained in section 1.1, shortest path algorithms attempt to determine paths of minimum total cost between vertices in a weighted graph. One of the most well-known shortest path algorithms is Dijkstra's algorithm, introduced by Edsger W. Dijkstra in 1956 [4]. Dijkstra's algorithm computes shortest paths from a source node s to all of the remaining and reachable nodes in a graph with nonnegative edge weights [4]. The exploring process is called edge relaxation. Relaxing an edge (u, v) attempts to improve the current best-known distance to v [4]. Therefore, there exists a path from a source node u to reach v with a tentative shortest distance. This update step can be written as

$$d(v) = \min(d(v), d(u) + w(u, v)).$$

Since Dijkstra's algorithm requires all edge weights to be nonnegative, on a weighted directed graph $G = (V, E)$ we assume that $w(u, v) \geq 0$ for each edge $(u, v) \in E$ [4]. The

algorithm begins by assigning every vertex an initial tentative distance of ∞ , except for the source vertex s , which is assigned distance 0. A min-priority queue Q stores vertices together with their current tentative distances, as implemented in Algorithm 1.

At each iteration, the algorithm removes the vertex u with smallest tentative distance from Q . Once a vertex is extracted from the priority queue, its shortest-path distance is considered finalized under the assumption of nonnegative edge weights. The algorithm then examines all adjacent vertices v . If traversing the edge (u, v) produces a shorter path to v , the tentative distance of v is updated and the pair $(d(v), v)$ is inserted into the queue. This process is known as edge relaxation, and continues until all reachable vertices have been explored.

The running time of the algorithm is

$$O((|V| + |E|) \log |V|),$$

when implemented using a binary heap priority queue.

Algorithm 1 Dijkstra's Algorithm

```

1: Choose source vertex  $s$ 
2: Initialize  $d(v) = \infty$  for all vertices  $v$ 
3:  $d(s) = 0$ 
4: Initialize priority queue
5:  $Q = [(0, s)]$ 
6: while  $Q \neq \emptyset$  do
7:    $(d, u) = \text{Extract-Min}(Q)$ 
8:   for each neighbor  $(v, w)$  adjacent to  $u$  do
9:     if  $d(v) > d(u) + w$  then
10:       $d(v) = d(u) + w$ 
11:      Insert  $(d(v), v)$  into  $Q$ 
12:     end if
13:   end for
14: end while

```

In contrast to the textbook formulation, which separates vertices into the finalized set S and non-finalized set $Q = V - S$, this implementation uses a priority queue-based approach without explicitly maintaining this distinction [3]. The classical formulation ensures that each vertex is processed exactly once upon finalization, whereas this heap-based implementation may contain multiple entries per vertex, allowing us to implicitly ignore outdated distance estimates when extracted.

Figure 1.5 below illustrates the execution of Dijkstra's algorithm, where the source vertex s is placed on the leftmost side of the graph [3]. Each vertex is labeled with its current estimate of the shortest-path distance, and shaded edges indicate which previous vertices are used to reconstruct shortest paths.

Graph (a) depicts the initial configuration before the algorithm begins processing, where the source vertex has distance 0 and all other vertices have distance ∞ [3]. At this point, the priority queue contains only the source vertex with its initial distance.

Graphs (b)-(f) depicts the step-by-step progression of the algorithm from the while loop as vertices are removed from the priority queue in order of smallest current distance estimate. At each step, the extracted vertex u is used to relax all outgoing edges (u, v) , which is intuitively the algorithm trying to improve its current best guess for one node v using another node u . Here, it is updating distance estimates when a shorter path is found and re-inserting updated entries into the priority queue again. Over time, the algorithm is essentially making distance estimates between nodes more accurate over time, with each "relaxation." The final subfigure (f) shows the resulting shortest-path distances and constructed parent-child structure after all reachable vertices have been processed or no shorter updates remain [3].

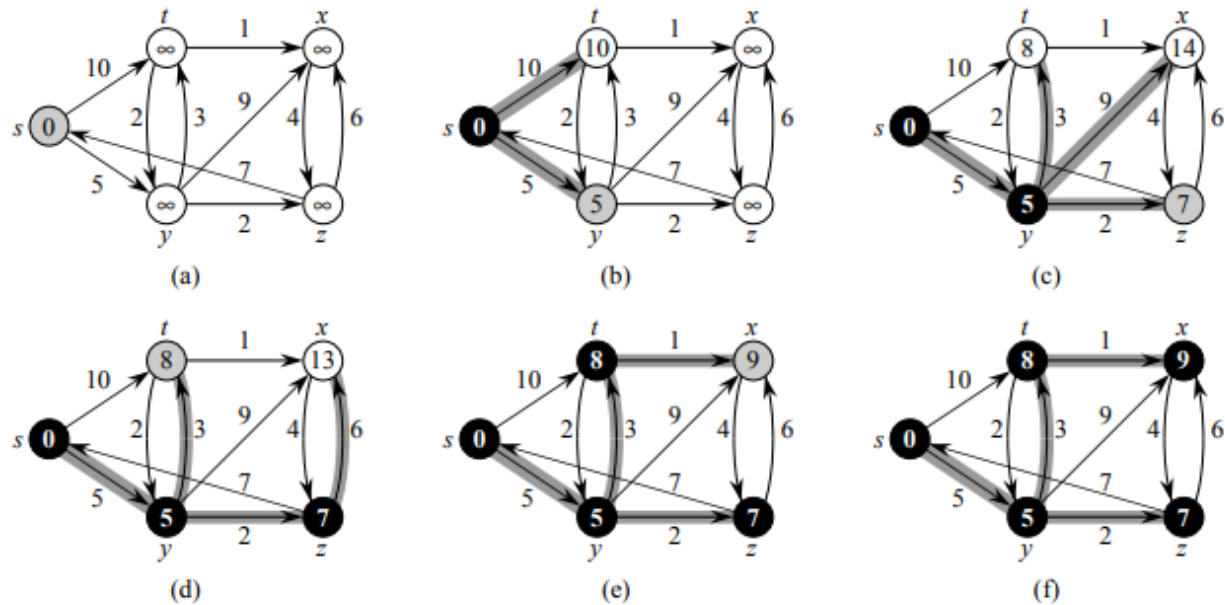


Figure 1.5: Dijkstra's algorithm [3]

1.2.2 Bellman-Ford Algorithm

Unlike Dijkstra's algorithm, the Bellman-Ford algorithm can handle graphs containing negative edge weights. Rather than greedily selecting the nearest unexplored vertex, Bellman-

Ford repeatedly relaxes all edges until shortest distances stabilize.

Given a weighted, directed graph $G = (V, E)$ with source node s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether or not there is a negative-weight cycle that is reachable from the source [3]. If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights [3].

The algorithm initializes all distance estimates to ∞ , except for the source vertex s , which is assigned distance 0. It then repeatedly relaxes all edges in the graph, where relaxing an edge (u, v) once again updates the tentative distance of v if a shorter path through u is found [3]. This relaxation step can be expressed as

$$d(v) = \min(d(v), d(u) + w(u, v)).$$

In the implementation used in this work (Algorithm 2), all edges are relaxed at most $|V| - 1$ times because any shortest path in a graph without cycles contains at most $|V| - 1$ edges [3]. To improve efficiency, the algorithm terminates early if no updates occur during an iteration [3].

Algorithm 2 Bellman-Ford Algorithm

```

1: Choose source vertex  $s$ 
2: Initialize  $d(v) = \infty$  for all vertices  $v$ 
3:  $d(s) = 0$ 
4: for  $i = 1$  to  $|V| - 1$  do
5:   updated = false
6:   for each edge  $(u, v) \in E$  do
7:     if  $d(u) + w(u, v) < d(v)$  then
8:        $d(v) = d(u) + w(u, v)$ 
9:       updated = true
10:    end if
11:  end for
12:  if updated = false then
13:    break
14:  end if
15: end for
    return  $d$ 

```

The running time of the algorithm is

$$O(|V||E|),$$

since each of the at most $|V| - 1$ iterations scans all edges once.

Figure 1.6 illustrates the execution of the Bellman-Ford algorithm on a directed weighted graph [3]. The source vertex s is initialized with distance 0, while all other vertices are assigned initial distance ∞ .

Graphs (a)–(e) show the state of the algorithm after successive passes over all edges. Unlike Dijkstra’s algorithm, which uses a priority queue to greedily select the next vertex with minimum tentative distance, the Bellman-Ford algorithm performs global relaxations over all edges in each iteration [3]. This means that instead of focusing on one vertex at a time, it systematically updates all distance estimates in parallel-like iterations. As such, vertices are not permanently selected or removed [3].

Because of this structure, Bellman-Ford does not require a greedy selection structure, such as a priority queue, and can correctly handle negative edge weights. In contrast, negative edge weights invalidate the greedy selection strategy used in Dijkstra’s algorithm [3].

In the final iteration (f), no further distance updates occur, indicating that the shortest-path estimates have stabilized [3]. In the implementation used here, the algorithm terminates early if a full pass produces no changes, since no further improvements are possible.

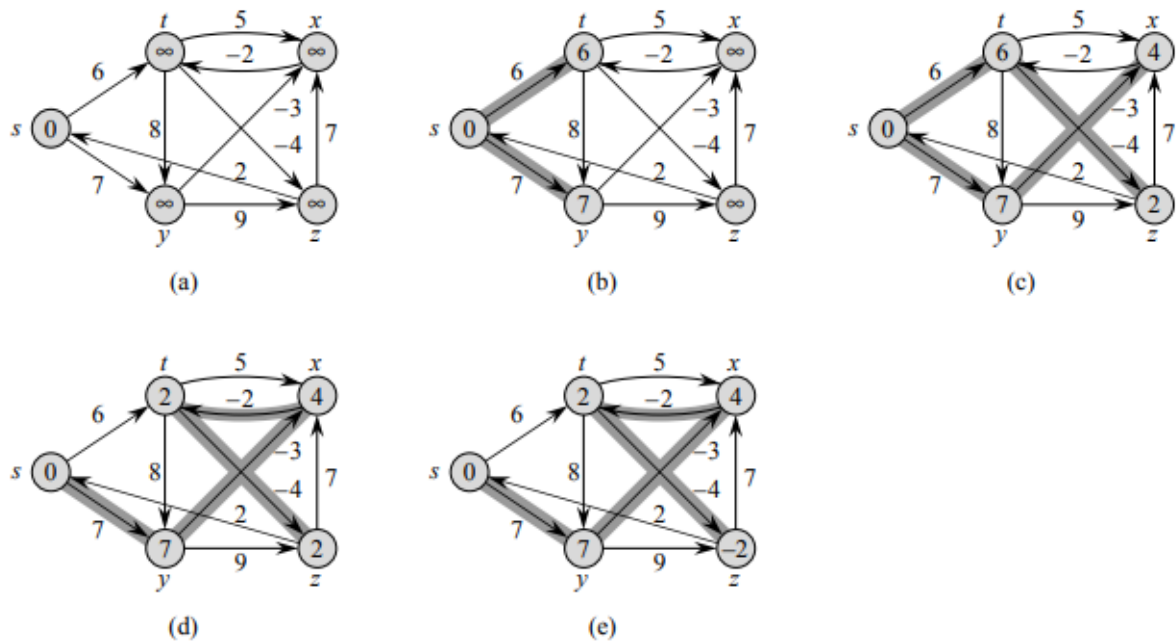


Figure 1.6: Bellman-Ford algorithm [3]

1.2.3 Comparison of Dijkstra and Bellman-Ford

Although both Dijkstra’s and Bellman-Ford algorithms solve the single-source shortest path problem, they differ significantly in strategy and applicability. Dijkstra’s algorithm uses a

greedy vertex-selection process and is generally more efficient, but it requires all edge weights to be nonnegative. In contrast, the Bellman-Ford algorithm repeatedly relaxes all edges without relying on a greedy selection structure, allowing it to correctly handle negative edge weights and detect negative-weight cycles. These differences illustrate the tradeoff between computational efficiency and generality in shortest path optimization problems.

1.3 Linear Programming and Optimization

Linear programming was a term introduced in the 1950s that originally referred to military plans or schedules for training, logistical supply, or deployment of men when computers were not commonly used [5]. Eventually, it started to be used for planning all sorts of economic activities, including transporting raw materials and products.

Despite what the name suggests, linear programming is not directly related to computer programming. The word linear refers to feasible plans restricted by linear constraints (inequalities), and that the quality of the plan (i.e., costs or duration) is measured by a linear function of the qualities being considered [5]. An example of a very simple linear programming problem (or linear program for short) is the following:

Maximize $x_1 + x_2$ subject to the constraints

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$x_2 - x_1 \leq 1$$

$$x_1 + 6x_2 \leq 15$$

$$4x_1 - x_2 \leq 10.$$

Each inequality defines a half-plane in $\mathbb{R}^2$, and the set of all points satisfying the constraints forms a convex polygon [5].

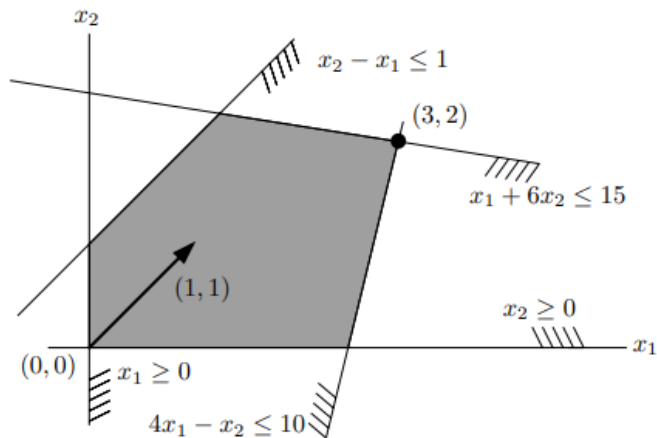


Figure 1.7: Polygon region [5]

The objective function $x_1 + x_2$ defines a family of parallel lines $x_1 + x_2 = c$, where larger values of c correspond to lines "farthest in the direction" of the vector $(1,1)$ [5]. Since the objective function is constant along lines perpendicular to $(1,1)$, translating these lines in the direction of the vector increases the objective value. The optimal solution is therefore the last point where one of these moving lines still intersects the feasible region while satisfying all constraints.

In this example, the optimal solution occurs at $(3,2)$, giving the maximum value

$$x_1 + x_2 = 3 + 2 = 5$$

. See Figure 1.8 for this illustration.

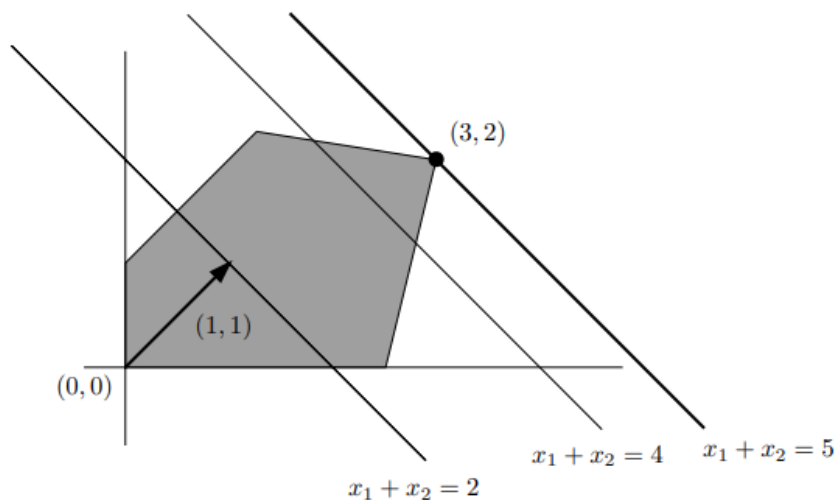


Figure 1.8: Moving lines[5]

In general, a linear program seeks to find a vector $x^* \in \mathbb{R}^n$ that maximizes or minimizes the value of a linear objective function subject to a system of linear constraints [5]. A standard linear programming formulation can be written as

$$c^T x = c_1 x_1 + \cdots + c_n x_n$$

where $c \in \mathbb{R}^n$ is a given vector. The linear equations and inequalities in the linear program are known as the constraints, and the number of constraints can be denoted by m [5]. Since a linear program is often written using matrices and vectors similar to the notation $Ax = b$, where A is a given $m \times n$ matrix and $c \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$ are vectors, each linear program can be written as

$$\text{maximize } c^T x$$

$$\text{satisfying } Ax \leq b,$$

Any vector satisfying the constraints is called a feasible solution, while a solution that optimizes the objective function is called an optimal solution [5].

Unlike combinatorial algorithms such as Dijkstra’s and Bellman–Ford, which are designed as fixed procedures with greedy update steps for specific graph problems, linear programming approaches the shortest-path problem by providing a general framework for modeling global optimization problems. Instead of prescribing a single step-by-step procedure, a linear program encodes a problem as a set of linear constraints and an objective function, and the solution is obtained through a general-purpose optimization method.

In this sense, combinatorial algorithms directly exploit graph structure to produce solutions with well-defined computational complexity. They construct a path incrementally by exploring vertices and edges, whereas linear programming searches over a continuous feasible region defined by linear constraints simultaneously. The performance of a linear programming solution therefore depends on the structure of the instance and the chosen solver, rather than a single fixed procedure.

A particularly important class of linear programs arises in network flow problems, where the variables represent flows along edges of a graph. In this setting, the linear constraints naturally encode conservation of flow at each node, making linear programming a natural framework for studying routing and shortest-path problems.

In a network flow formulation, each directed edge $(u, v) \in E$ is assigned a variable x_{uv} representing the amount of flow sent from node u to node v [6]. Each edge is also associated with a cost c_{uv} , representing the cost per unit of flow along that edge [6]. The objective is

to minimize the total cost of routing flow through the network:

$$\min \sum_{(u,v) \in E} c_{uv} x_{uv}.$$

To define the constraints, we first describe the total flow entering and leaving a node $v \in V$. The total flow into node v is given by

$$\sum_{(u,v) \in E} x_{uv},$$

while the total flow out of node v is given by

$$\sum_{(v,w) \in E} x_{vw}.$$

Flow conservation requires that at each node in the network, the total incoming flow equals the total outgoing flow. This ensures that flow is neither created nor destroyed at all other vertices, but rather transported through the network from source to destination. At the source and sink, the net imbalance is fixed to enforce a unit flow from s to t [6].

$$\sum_{(u,v) \in E} x_{uv} - \sum_{(v,u) \in E} x_{vu} = \begin{cases} 1 & \text{if } v = s \\ -1 & \text{if } v = t \\ 0 & \text{otherwise} \end{cases}$$

Finally, flow along each edge must be nonnegative:

$$x_{uv} \geq 0 \quad \forall (u,v) \in E,$$

since negative flow would contradict the directed nature of the edges. This could be called the nonnegativity constraints.

Chapter 2

Experimental Design

The following section presents the experimental methodology used to compare combinatorial shortest-path algorithms with a linear programming formulation of the shortest-path problem. The experiments were designed to evaluate differences in runtime performance, computational cost, and scalability across varying graph sizes.

All experiments were conducted within a Github-based Passagemath workspace as part of the UC Davis Math Lab undergraduate research program in polyhedral geometry and optimization. The work was completed as part of an undergraduate thesis project. Within this environment, simplified Python implementations of Dijkstra’s and Bellman-Ford algorithms were developed for direct experimental comparison and runtime analysis. These implementations operated on weighted directed graphs represented as adjacency lists and explicitly performed the relaxation procedures described in Section 1.

For the optimization-based approach, the shortest-path problem was formulated as a linear program using PassageMath’s `MixedIntegerLinearProgram` framework. Rather than implementing an LP solver manually, the project focused on constructing the objective function and flow conservation constraints corresponding to the shortest-path formulation, while relying on PassageMath’s built-in optimization backend to compute solutions. Randomly generated weighted directed graphs were then used to compare the computational behavior of the combinatorial and optimization-based approaches.

2.1 PassageMath Environment

2.2 Algorithm Implementations

2.3 Runtime and Scalability Comparisons

Chapter 3

Results and Discussion

Chapter 4

Future Work

Bibliography

- [1] R. A. Brualdi, *Introductory Combinatorics*, 5th edition, Pearson Prentice Hall, 2009.
- [2] L. Lovász, J. Pelikán, K. Vesztergombi, *Discrete Mathematics: Elementary and Beyond*, 2nd edition, Springer, 2019.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd edition, MIT Press, 2009.
- [4] H. Ortega-Arranz, D. R. Llanos, A. Gonzalez-Escribano, *The Shortest-Path Problem*, Synthesis Lectures on Theoretical Computer Science, Springer, Cham, 2015. doi:10.1007/978-3-031-02574-7
- [5] J. Matoušek, B. Gärtner, *Understanding and Using Linear Programming*, Springer, Berlin, 2007.
- [6] R. J. Vanderbei, *Linear Programming: Foundations and Extensions*, 4th edition, Springer, New York, 2014.